3. SDLC and Delivery Frameworks (Run) 🏃

This section moves from the planning mechanics of the "Walk" phase to the dynamic, iterative execution of "Run." As a Project Manager in an IT outsourcing environment, you must master various Software Development Life Cycles (SDLC) and delivery frameworks to manage client expectations while protecting team health and project margins. This module provides a deep dive into the engineering, scaling, and flow principles that move a project from a Statement of Work (SOW) to a successful production release.

3.1. The Agile Mindset and Principles

Agile practices advocate for adaptive planning, incremental development, early delivery, continual improvement, and collaborative work with clients and end users. In outsourcing, this mindset is the most effective defense against the "obsolescence trap," where a product becomes irrelevant during a long development cycle because the client's market shifted while the vendor was stuck in a traditional, rigid process.

3.1.1. Agile Manifesto Values and Principles

The Agile Manifesto serves as the bedrock for modern delivery, establishing four core values that prioritize human interaction over rigid, contract-heavy structures:

- **Individuals and interactions over processes and tools:** Skilled engineers collaborating directly with client stakeholders deliver more value than a team strictly following a generic workflow.
- **Working software over comprehensive documentation:** The primary measure of progress for the client is functional code, not a 200-page specification document that no one reads.
- **Customer collaboration over contract negotiation:** Instead of pointing to the SOW every time a detail changes, the PM works with the client as a partner to discover requirements.
- **Responding to change over following a plan:** The delivery team remains ready to adjust the roadmap when the client's business priorities or market conditions shift.

Real-Life Example: The Pivot Request. A vendor was building a travel booking engine for a client when a global market shift halted travel. Instead of sticking to the original plan and delivering a product no one would use, the PM worked with the client to pivot the scope toward a cancellation and refund portal. This response to change preserved the client's business value and the vendor's reputation.

3.1.2. Adaptive vs. Predictive Planning

Predictive planning assumes that requirements can be fully understood upfront. This is often requested in outsourcing for fixed-price contracts to minimize perceived financial risk. Adaptive planning acknowledges that uncertainty requires constant adjustment through short sprints, contrasting sharply with traditional timescales that stretch into months or years. In complex outsourcing engagements, adaptive planning is necessary to survive when the client's "As-Is" state is poorly documented.

3.1.3. Lean Thinking and Principles

Lean principles focus on optimizing the entire value stream by identifying and removing inefficiency. There are seven fundamental principles in Lean software development:

- **Eliminate waste:** Identify and remove anything that does not add value to the customer, such as defects, partially done work (WIP), or unnecessary features that weren't requested.
- **Build quality in:** Integrity is maintained throughout the process rather than being verified only during a final QA phase.
- **Amplify knowledge:** Teams should prioritize continuous learning and the sharing of insights to avoid redundant work.
- **Delay commitment:** Decisions should be made at the "Last Responsible Moment" to ensure they are based on the most current client data.
- **Deliver fast:** Shortening the cycle from request to deployment increases the frequency of client feedback.
- **Respect people:** Empower the engineering team and trust their technical expertise.
- **Optimize the whole:** Focus on the system's total flow rather than just optimizing the speed of one department while others are overloaded.

Real-Life Example: The Automation Strategy. An outsourcing PM noticed that the client's manual security approval process was causing two-week delays between development and testing. By applying the eliminate waste principle, the PM convinced the client to move to automated security scans for low-risk changes. This reduced the idle time of the developers and significantly increased the overall delivery speed of the vendor team.

3.2. Scrum in Practice

Scrum is an empirical framework designed to help teams deliver value through iterative solutions. It is structured around specific accountabilities, artifacts, and events.

3.2.1. Scrum Accountabilities and Events

As of the November 2020 update, the Scrum Guide uses the term "accountabilities" instead of "roles" to emphasize ownership:

- **Scrum Master:** Accountable for the team's effectiveness and for establishing Scrum as defined in the guide.
- **Product Owner:** Accountable for maximizing the value of the product and effectively managing the Product Backlog.
- **Developer:** The people committed to creating any aspect of a usable Increment each Sprint.

The framework utilizes five formal events for inspection and adaptation:

- **The Sprint:** The container event for all other activities, typically lasting one month or less.
- **Sprint Planning:** Where the team defines what can be delivered and how that work will be achieved.
- **Daily Scrum:** A 15-minute daily event for developers to inspect progress toward the Sprint Goal.
- **Sprint Review:** An event to inspect the outcome of the Sprint and determine future adaptations with stakeholders.
- **Sprint Retrospective:** An event for the team to plan ways to increase quality and effectiveness.
- **Backlog Refinement:** Also known as grooming, these sessions are essential for reviewing and ordering items to ensure they are ready for future Sprints.

3.2.2. Story Points: Complexity and Relative Estimation

Software development is inherently abstract, making absolute time-based estimation inaccurate. In outsourcing, we use story points to provide reliable forecasts while protecting the team from micromanagement.

- **Weber's Law and Relative Estimation:** Humans are scientifically better at comparing the relative size of things than guessing absolute values. Story points use the Fibonacci scale because our perception of change is proportional to the original size of the task.
- **Effort vs. Complexity:** A story point represents the total effort and complexity, not just the literal time needed to satisfy the Definition of Done.

- **Story Points vs. Ideal Hours:** Hours are personal—a senior engineer takes less time than a junior. Story points provide a team-level baseline. This allows the PM to measure velocity and provide the client with a predictable release roadmap.
- **Planning Poker:** This technique is used for collective estimation to reach consensus and avoid anchoring bias.

Real-Life Example: The Estimation Gap. A client demanded to know exactly how many hours a complex migration would take. The PM used planning poker and the team assigned it 13 story points, indicating high uncertainty. By explaining that 13 points was the size and the team's velocity was 20 points per sprint, the PM set a realistic expectation that this migration would consume most of one sprint. This was more accurate than guessing a specific number of hours that would have been exceeded.

3.2.3. Sprint Related Metrics

Measuring agility requires specific metrics to distinguish team performance from overall project health:

- **Velocity:** The average story points delivered across a given time period.
- **Capacity:** The estimated total amount of engineering time available for the upcoming sprint.
- **Sprint Goal Achievement:** Tracking if the core purpose of the sprint was met.
- **Work Breakdown:** Monitoring the ratio of technical debt versus bug-fixes versus feature development.
- **Sprint Burndown Chart:** A visual guide showing work left to do versus time in the sprint.
- **Commitment Reliability:** A comparison of committed story points at planning versus those completed.
- **Sprint Scope Change:** Tracking points or hours added or removed mid-sprint.
- **Backlog Health:** Measuring how many future sprints have been planned and estimated.

Real-Life Example: The Silent Blocker. During the daily scrum, a developer reports no blockers. However, the sprint burndown chart shows a flat line for three days indicating no progress. The scrum master investigates and finds the developer is waiting for an API key from the client's internal team. The issue is raised, the key is procured, and the burndown starts moving down again.

3.3. Kanban and Flow Management

Kanban aims to visualize the entire development process from start to finish and eliminate bottlenecks.

3.3.1. Visualization and WIP Limits

The core of Kanban is managing the value stream through clear visibility.

- **Work In Progress (WIP):** The number of tickets in a specific status that have been started but not finished.
- **WIP Limits:** Constraints applied to columns to prevent the team from starting new work before finishing current tasks, highlighting process bottlenecks immediately.

3.3.2. Kanban Flow Metrics and Reports

Predictability in Kanban is achieved through data-driven metrics:

- **Throughput:** The number of tickets finished per unit of time.
- **Cycle Time:** The amount of time elapsed between starting work on a ticket and finishing it.
- **Lead Time:** The total time from when a task was created until its final completion.
- **SLAs and Service Classes:** Service Line Agreements define expected delivery times for different work categories.
- **Jira Reports:** Essential visualization tools including the Cumulative Flow Diagram (CFD) to identify bottlenecks and the Control Chart to track cycle time stability.

Real-Life Example: The QA Logjam. A Kanban team's board showed thirty tickets in the ready for QA column. The CFD showed the testing band was extremely thin while progress was widening. The PM realized the developers were finishing work faster than QA could verify it. They imposed a developer WIP limit, forcing developers to stop coding and help the QA team with test automation. This cleared the logjam and restored flow.

3.4. Plan-based and Hybrid Frameworks

While Agile is often preferred for uncertainty, traditional models remain critical for specific industries and project types.

3.4.1. Waterfall Phases and Applicability

Waterfall breaks a project into linear sequential phases where each phase depends on the previous deliverables:

1. **Requirements**.
2. **Design**.
3. **Implementation**.
4. **Verification**.
5. **Maintenance**.

Waterfall is best suited for small projects without change management needs, well-known requirements, or medical, space, and military programs where safety outweighs agile speed.

Real-Life Example: The Banking Migration. An outsourcing team was hired to migrate a bank's core database. Because the regulatory requirements were fixed and any error could cause massive financial loss, the client mandated a waterfall approach. Each requirement was signed off by the bank's legal team before a single line of code was written. This provided the high level of control and auditability the client required.

3.4.2. V-Model Advantages

The V-model is a modification of Waterfall created around 2005 that adds validation and verification steps for every stage. It has features similar to PRINCE2 and provides several advantages:

- **Minimization of project risks:** Potential issues are identified earlier through rigorous verification.
- **Improvement of quality:** Every phase has a corresponding testing or validation step.
- **Reduction of project costs:** Catching defects early in the cycle is significantly cheaper than fixing them in production.
- **Stakeholder Communication:** It provides a clear structure for technical and business alignment at each stage.

Real-Life Example: Automotive Safety Software. An automotive client used the V-model to build braking software. For every line of requirement, there was a corresponding test case in the verification wing of the V-shape. When a requirement was found to be contradictory during testing, the team traced it back immediately to the design phase, preventing a potentially fatal failure in the physical car.

3.4.3. Hybrid Delivery Models

Hybrid frameworks combine elements of plan-based and agile frameworks to satisfy specific contractual or operational needs:

- **Water-Scrum-Fall:** Initiation and planning use Waterfall for budgeting and high-level requirements, execution follows Scrum for development, and final delivery/acceptance is Waterfall.
- **Scrumban:** A mix of Scrum and Kanban useful for teams struggling to adopt full Scrum ceremonies or for maintenance projects.
- **ScrumXP:** A combination of Scrum management and Extreme Programming engineering practices often used in scaled environments like SAFe.

Real-Life Example: The Fixed-Price Agile. A client signed a fixed-price SOW for a new CRM. The PM used waterfall for the initial discovery and design phase to secure the budget and baseline requirements. Once the design was signed off, the delivery team switched to Scrum for the actual build. This hybrid approach gave the client financial predictability and the vendor execution flexibility.

3.5. Extreme Programming (XP) and Agile Techniques

XP provides the engineering practices that ensure quality in agile delivery.

- **XP Core Practices:** Includes Pair Programming, Continuous Integration (CI), Test-Driven Development (TDD), Refactoring, Coding Standards, and Collective Code Ownership.
- **Refactoring:** Continuously improving the internal design of the code without changing its external behavior.
- **Sustainable Pace:** Maintaining a workload, typically a 40-hour working week, that can be sustained indefinitely without causing developer burnout.
- **Story Mapping:** A visual technique for building the product backlog by arranging user activities in a horizontal flow.
- **BDD:** Behavior-Driven Development where tests are based on behavior scenarios to ensure the software meets business goals.

Real-Life Example: TDD in Financial Systems. A vendor team was struggling with bugs in a complex tax calculation engine. They implemented test-driven development, writing the expected results as tests before coding the logic. This forced them to handle edge cases upfront. As a result, production defects dropped by seventy percent and the client gained full confidence in the system's accuracy.

3.6. Scaled Agile Frameworks

When delivery involves dozens of teams and hundreds of people, standard frameworks must be scaled to ensure synchronization.

3.6.1. Scrum of Scrums (SoS)

Created in 1996, this is the most light-weight scaling method. It introduces specialized roles and events to manage interdependencies:

- **Chief Product Owner:** Oversees individual Product Owners to ensure a unified product vision.
- **Scrum of Scrum Master:** Helps teams work efficiently by managing priorities and removing impediments across the entire program.
- **Scaled Daily Scrum:** An event where team representatives discuss impediments, risks, and cross-team dependencies.

Real-Life Example: Synchronizing Three Teams. An outsourcing vendor had three Scrum teams building a web portal, a mobile app, and a backend API. They used Scrum of Scrums to coordinate releases. When the backend team was delayed, the Scrum of Scrum Master identified the risk to the mobile team early. This allowed the mobile team to work on offline features while waiting, preventing a total project stall.

3.6.2. Scrum at Scale (S@S)

A framework focusing on "minimum viable bureaucracy" and scaling small teams of 3-9 people. It utilizes:

- **Chief Product Owner (CPO):** Generates a single backlog for all teams and aligns priorities with stakeholders.
- **Scrum of Scrums Master (SoSM):** Responsible for the joint efforts of all Scrum Masters.
- **Executive Action Team (EAT):** Responsible for the Scrum transformation and removing organizational blockers.
- **Executive MetaScrum Team (EMT):** Owns the organizational vision and decides on product restructuring or retirement.

Real-Life Example: Enterprise Agile Transformation. A global payments company, Worldline, used Scrum at Scale to modernize its operations. They formed an Executive Action Team to remove corporate-level blockers like slow procurement processes. This

allowed their multiple delivery teams to move as fast as a startup while maintaining enterprise-level security and compliance.

3.6.3. Scaled Agile Framework (SAFe)

SAFe is the most popular scaling framework, created in 2011. It offers four versions: Essential, Large Solution, Portfolio, and Full SAFe.

- **Team Level:** This is the first layer where the PO, SM, and Dev Team work on a Team Backlog using Stories and Enabler Stories (infrastructure, architecture, compliance).
- **Program Level:** Teams combine into an Agile Release Train (ART) led by a Release Train Engineer (RTE). Other roles include System Architect, Product Managers, and Business Owners.
- **Program Backlog:** Consists of Features and Enabler Features, which are decomposed to stories for the teams.
- **PI Cycle:** Work happens in a Program Increment (PI) typically lasting 8-12 weeks. A PI consists of 5 iterations: 4 development iterations with System Demos and 1 Innovation and Planning (IP) iteration.
- **IP Iteration:** Includes a buffer for leftover work, the PI Planning event, and the final Inspect and Adapt demo.
- **Prioritization:** Uses Weighted Shortest Job First (WSJF), calculated as Cost of Delay divided by Job Duration.

Real-Life Example: The Global ART. Radware, a global cybersecurity leader, implemented SAFe to align multiple product lines across global sites. By launching Agile Release Trains and conducting regular PI Planning, they eliminated handoff delays and misalignments typical of their previous sequential process. This synchronization allowed them to respond to new security threats 30% faster than before the transformation.

3.6.4. Large-Scale Scrum (LeSS)

LeSS is a Scrum-based customer-centric framework created in 2005. It has two versions: Basic LeSS for 2-8 teams (up to 50 people) and LeSS Huge for larger setups.

- **Feature Teams:** Crucial cross-functional, cross-component full-stack teams that work together in a shared code environment to create "done" items.
- **Structure:** Includes one Product Owner for all teams and a Scrum Master for 1-3 teams.

Real-Life Example: Descaling for Speed. Ericsson adopted LeSS for its R&D development program. They transformed their component-based silos into cross-functional feature teams. By having one Product Owner and one Product Backlog for up to 15 teams, they simplified their organization and achieved a continuous feature delivery model that was impossible under their previous plan-driven process.

3.6.5. Nexus

Nexus is a framework from Scrum.org designed for scaling Scrum involving 3-9 teams.

- **Nexus Integration Team:** Includes the Product Owner, Scrum Masters, and members from teams, responsible for the integrated increment and the overall result.
- **Artifacts:** A single Nexus Product Backlog for all teams and individual Nexus Sprint Backlogs. It delivers one Integrated Increment.
- **Events:** Includes Nexus Sprint Planning, Daily Scrum for the Integration Team, and a single Nexus Sprint Review and Retrospective for the whole group.
- **Cross-Team Refinement:** Sessions where teams manage the backlog collaboratively to identify and resolve dependencies.

Real-Life Example: Integration First. Pentacomp used the Nexus framework to develop an e-health prescription platform for the Ministry of Health in Poland. With nearly 100 developers across 9 teams, they faced massive integration challenges. By forming a Nexus Integration Team and synchronizing through Nexus events, they delivered a shippable product that became critical during the pandemic, allowing patients to get prescriptions without physical visits.

3.6.6. The Spotify Model and UnFix

- **Spotify:** A cultural model focusing on autonomy. It uses terms like Squads (dev teams), Tribes (40-150 people), Chapters (technology specialists within a tribe), and Guilds (communities of interest). A Tribe Trio consisting of a Tribe Lead, Product Lead, and Design Lead aligns the work.
- **UnFix:** A pattern library for organizational design that allows companies to build flexible, tailored structures rather than adopting a fixed framework.

Real-Life Example: Squad Autonomy. A digital music company organized its vendor engineers into squads. Each squad was given a mission, such as make search faster, and the autonomy to choose their own tools. To prevent the teams from becoming silos, they

created chapters for all developers to share coding tips. This allowed the company to innovate quickly while maintaining high technical standards across the organization.